```
void thread_a()                                    ⑥
{
    high_level_func();
}

hierarchical_mutex other_mutex(100);               ⑦
void do_other_stuff();

void other_stuff()
{
    high_level_func();                             ⑧
    do_other_stuff();
}

void thread_b()                                    ⑨
{
    std::lock_guard<hierarchical_mutex> lk(other_mutex);    ⑩
    other_stuff();
}
```

`thread_a()` ❻ abides by the rules, so it runs fine. On the other hand, `thread_b()` ❾ disregards the rules and therefore will fail at runtime. `thread_a()` calls `high_level_func()`, which locks the `high_level_mutex` ❹ (with a hierarchy value of 10000 ❶) and then calls `low_level_func()` ❺ with this mutex locked in order to get the parameter for `high_level_stuff()`. `low_level_func()` then locks the `low_level_mutex` ❸, but that's fine because this mutex has a lower hierarchy value of 5000 ❷.

`thread_b()` on the other hand is *not* fine. First off, it locks the `other_mutex` ❿, which has a hierarchy value of only 100 ❼. This means it should really be protecting ultra-low-level data. When `other_stuff()` calls `high_level_func()` ❽, it's thus violating the hierarchy: `high_level_func()` tries to acquire the `high_level_mutex`, which has a value of 10000, considerably more than the current hierarchy value of 100. The `hierarchical_mutex` will therefore report an error, possibly by throwing an exception or aborting the program. Deadlocks between hierarchical mutexes are thus impossible, because the mutexes themselves enforce the lock ordering. This does mean that you can't hold two locks at the same time if they're the same level in the hierarchy, so hand-over-hand locking schemes require that each mutex